Browser-Based Gesture Control for Multi-Device Smart Home Appliances Using IoT

Aun Shahid, M. Usman Baig, Shahnawaz Ali Ashgar and Umar Hanif
Military College of Signals, National University of Sciences and Technology
Rawalpindi, Pakistan
{ashahid.bse22mcs, mbaig.bse22mcs, sali.bse22mcs, uhanif.bse22mcs}@student.nust.edu.pk

***Abstract*—Conventional smart home interfaces—physical switches, vendor applications, and voice assistants—require users to locate and address a device explicitly before issuing a command, imposing unnecessary interaction steps when the target appliance is already visible in the room. This paper presents a browser-based gesture control system that eliminates this lookup step by treating the live camera view as the interaction surface. The proposed architecture integrates a React frontend, TensorFlow.js Handpose for 21-point hand landmark estimation, Fingerpose for gesture classification, a YOLOv11 object detector served through FastAPI for appliance identification, and Blynk cloud commands for IoT actuation via an ESP32 relay module. A spatial selection mechanism extends the design beyond single-device control to multiple concurrently visible appliances: the frontend computes bounding box centers from YOLO detections, sorts them by horizontal position in the camera frame, assigns each a dedicated Blynk virtual pin, and routes each gesture command to the appliance whose bounding box center is nearest the detected wrist landmark. Appliances outside the targeted spatial region receive no update and remain in their current state. Two gestures are supported: thumbs-up writes a logic-high value to the target pin and thumbs-down writes logic-low. Functional testing across single-device and two-device configurations confirmed correct relay switching and successful spatial discrimination between simultaneously powered bulbs. The results demonstrate that browser-based computer vision and cloud IoT actuation can be unified into a compact, installation-free, touch-free interaction pipeline without dedicated gesture hardware.**

***Index Terms*—Internet of Things, smart home, gesture recognition, TensorFlow.js, YOLO, spatial selection, ESP32, Blynk, hand landmark estimation**

I. Introduction

The Internet of Things (IoT) connects physical devices to software services so that sensing, communication, and actuation can be coordinated over a network [1]. In residential environments, this model is widely applied to lighting, climate control, and energy management. Despite the maturity of the enabling infrastructure, the dominant interaction paradigms for smart home appliances remain touch-based mobile interfaces, physical switches, and voice commands [2]. These methods are practical when the user is distant from the target device, but they impose an unnecessary device-lookup step when the appliance is already within the user's line of sight.

Camera-based interaction addresses this gap by collapsing the lookup step: the camera view serves as both the observation surface and the command interface [3]. Azuma defined augmented reality systems as those that combine real and virtual content, operate interactively in real time, and register virtual content with the physical world. The architecture described in this paper follows that principle at a practical scale: a YOLOv11 detector identifies which appliances are present in the live camera frame, and the user issues a hand gesture spatially directed toward a detected appliance to trigger the corresponding IoT command.

Prior gesture-based smart home systems demonstrate the feasibility of this interaction model, but most rely on server-side inference for both object detection and gesture classification [4], [5]. Moving classification into the browser removes the requirement for a dedicated recognition backend and reduces the deployment footprint to a single web page [6]. However, shifting inference client-side introduces a second challenge: when multiple appliances appear simultaneously in the camera frame, the system must unambiguously route each gesture to the intended target. Routing a command to the wrong device is both a usability failure and, in the context of mains-voltage switching, a safety concern.

This paper addresses that challenge with a spatial selection mechanism that associates each detected appliance bounding box with a unique Blynk virtual pin and routes commands exclusively to the appliance nearest the user's wrist position in the camera coordinate space. The main contributions are:

- A browser-based gesture control architecture that integrates YOLO11 appliance detection, TensorFlow.js Handpose landmark estimation, Fingerpose classification, and Blynk IoT actuation within a single React application, requiring no native installation on the user's device.
- A spatial selection mechanism that extends single-device gesture control to an arbitrary number of concurrently visible appliances while guaranteeing that unselected devices remain unaffected by each command.
- A gated control design in which gesture recognition is activated only after the appliance detector confirms a target is present in the current frame, suppressing false commands when no valid target exists.
- A functional evaluation of the complete pipeline across both single-device and two-device configurations, including observations on spatial discrimination performance.

II. Related Work

A. Augmented Interaction and Spatial Command Models

Milgram and Kishino introduced the reality-virtuality continuum to classify systems that overlay virtual information on physical environments [7]. Smart home interfaces benefit from this spatial coupling because co-locating a virtual command with the physical appliance it controls reduces cognitive load and eliminates the device-lookup step inherent to menu-based interfaces. The present work makes this coupling explicit: each YOLO bounding box anchors a virtual command channel to the physical device it encloses, and the user selects a target implicitly by positioning their hand near it.

B. Gesture-Based Smart Home Control

Gesture recognition has been applied to smart home control across a range of hardware and software configurations. Yang et al. developed an IoT-enabled system in which camera-captured hand gestures control relay-connected appliances, reporting command accuracy above 90% under controlled lighting conditions [4]. Alabdullah et al. proposed a markerless classification technique using feature fusion and a recurrent neural network, achieving 95.4% accuracy across ten gesture classes without physical markers or wearable sensors [5]. Liu et al. demonstrated gesture control of smart home devices using MediaPipe hand tracking on an Android platform with sub-50 ms command latency [8]. Sideridis et al. evaluated IMU-based gesture recognition for IoT environments using recurrence quantification analysis to detect gesture onset without explicit markers, reporting robust performance across six gesture classes [9].

A common design choice in the above systems is server-side classification: frames are transmitted from the client to a backend inference engine, which returns the recognized gesture. This approach provides access to high-capacity models but introduces network round-trip latency and requires a persistent server connection. The present prototype shifts both classification and spatial reasoning into the browser, using TensorFlow.js and Fingerpose, reducing the backend to a lightweight appliance detection service.

C. In-Browser Machine Learning

TensorFlow.js enables deployment of trained neural network models directly in the browser using WebGL-accelerated inference [6]. The Handpose model estimates 21 three-dimensional hand landmarks from a single RGB frame, providing sufficient geometric information for rule-based gesture classification without transmitting video to a remote server. Fingerpose encodes gestures as sets of finger curl and direction descriptors evaluated against the landmark array, yielding a scored match for each registered gesture [10]. This combination makes real-time hand tracking practical in any browser without native code, driver installation, or specialized depth sensors.

D. Object Detection for Appliance Awareness

Redmon et al. introduced YOLO as a unified single-pass framework for bounding box regression and class prediction, enabling real-time object detection [11]. Jocher et al. extended this line with YOLOv8 and YOLO11, improving small-object fidelity through an anchor-free decoupled head and compact C3k2 bottleneck modules [12]. Serving the detector through a local FastAPI endpoint decouples GPU-intensive inference from the browser thread, allowing the React application to query detection results asynchronously without blocking the gesture recognition loop.

E. Cloud IoT Actuation

The Blynk platform provides a cloud IoT abstraction layer that bridges HTTP requests from the frontend to GPIO events on microcontroller hardware [13]. Its HTTPS datastream API accepts device token, virtual pin, and value fields in a single GET request, making it straightforward to map gesture classifications to actuator commands without custom firmware beyond standard Blynk library usage. The ESP32-WROOM-32 module is well-suited to this role because it integrates dual-core processing, WiFi, and configurable GPIO in a compact, widely available package [14].

III. System Design

A. Design Requirements

Four requirements governed the design. First, the interface must operate in a standard browser, imposing no native application installation on the user's device. Second, the live webcam feed must serve as the sole input medium, making the camera view the complete interaction surface. Third, high-voltage switching must be executed by dedicated relay hardware under microcontroller supervision, isolating the browser from mains circuitry entirely. Fourth, the probability of unintended actuation must be minimized both by gating gesture recognition on a confirmed appliance detection and by constraining each command to the spatially nearest target.

B. System Architecture

Fig. 1 illustrates the complete end-to-end pipeline. The React frontend acquires webcam frames through `react-webcam`. Every 3 s, a JPEG snapshot is base64-encoded and transmitted via HTTP POST to a local FastAPI service at `/predict/image`. The service decodes the payload, performs YOLO11 inference, and returns a JSON list of detections, each carrying a class label, a confidence score, and bounding box pixel coordinates. The frontend retains all detections whose label matches `bulb`.

When at least one valid bulb bounding box is stored, gesture recognition becomes active. TensorFlow.js Handpose estimates 21 three-dimensional landmarks from the current video frame on each render cycle. Fingerpose evaluates the landmark array against a predefined gesture descriptor set and returns the highest-scoring gesture above the acceptance threshold. If the recognized gesture is `thumbs_up` or `thumbs_down`, the spatial selection mechanism described in Section III-C identifies the target index k, and a Blynk HTTPS request is dispatched to virtual pin Vk with the corresponding binary value.

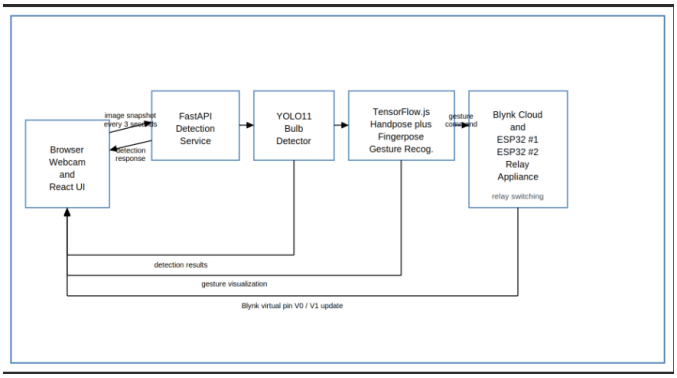

Fig. 1. End-to-end pipeline: React frontend polls the FastAPI detector every 3 s, activates gesture recognition upon bulb detection, applies spatial selection to identify the target virtual pin, and dispatches the Blynk actuation command.

C. Multi-Appliance Spatial Selection

Fig. 2 illustrates the spatial selection procedure for two simultaneously detected bulbs. Upon receiving a positive detection response, the frontend computes the center pixel of each bounding box, sorts the boxes in ascending order of horizontal coordinate c_x, and assigns virtual pin indices sequentially: V0 to the leftmost bulb, V1 to the next, and so on. This ordering is deterministic within a frame and remains stable across frames provided the relative horizontal positions of the bulbs do not change.

When a valid gesture is recognized, the wrist landmark (Handpose landmark 0) position (w_x, w_y) in camera pixel coordinates is compared against each stored bounding box center. The bulb index k with the minimum Euclidean distance is selected as the active target, and the Blynk request is issued only to Vk. No update is sent to any other virtual pin, leaving all remaining appliances in their current state regardless of the command value.

This mechanism scales to any number of simultaneously visible appliances without requiring changes to the gesture vocabulary, hardware wiring, or firmware. Each additional controlled appliance requires one relay channel and one virtual pin in the Blynk project configuration.

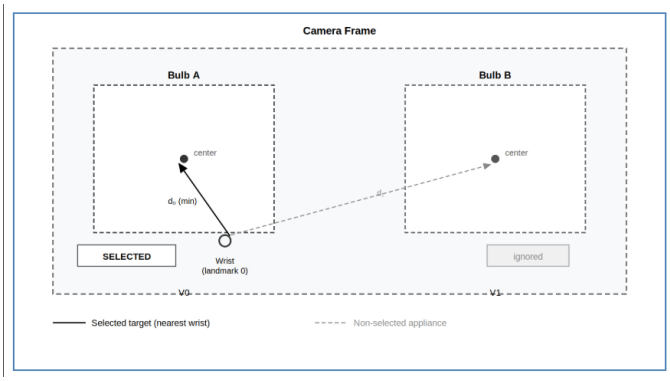

Fig. 2. Spatial selection with two detected bulbs. Bounding boxes are sorted left-to-right and assigned pins V0 and V1. The wrist landmark is nearer to Bulb A; consequently, only pin V0 is updated. Bulb B receives no command.

D. Hardware Layer

The physical layer comprises one ESP32-WROOM-32 development module, one single-channel relay module, and one standard LED bulb per controlled appliance. Each ESP32 connects to the Blynk project over WiFi and monitors its assigned virtual pin. A logic-high update closes the relay and completes the bulb circuit; a logic-low update opens it. High-voltage switching is confined entirely to the relay and bulb wiring, maintaining complete electrical isolation from the ESP32 logic circuitry and from the browser application.

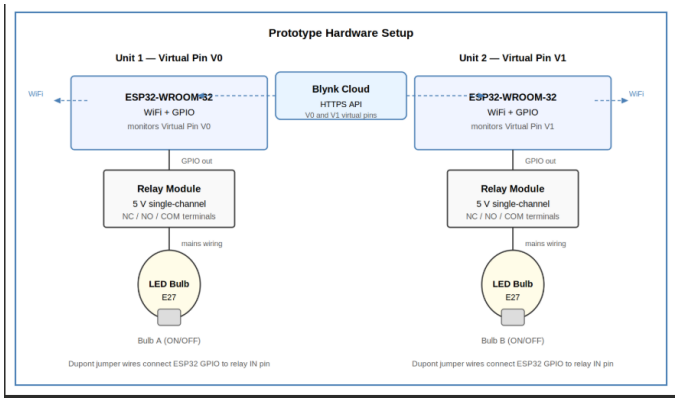

Fig. 3. Prototype hardware: two ESP32 modules, relay modules, and controlled bulbs. Each bulb connects to a dedicated relay channel and is assigned a unique Blynk virtual pin.

E. Software Stack

Table I summarizes the implemented components. The frontend is built on React 17 with TensorFlow.js 3.12, TensorFlow Handpose 0.0.7, Fingerpose 0.1.0, and `react-webcam`. The backend uses FastAPI with the Ultralytics YOLO11 Python package [15]. The frontend subsumes video capture, bounding box management, spatial selection logic, gesture classification, and command dispatch. The backend is responsible solely for image decoding and YOLO inference, keeping the server stateless and independently replaceable.

IV. Implementation

A. Appliance Detection Backend

The detection service accepts an HTTP POST request to `/predict/image` carrying a JSON body with a base64-encoded image string. The server strips the data-URL prefix when present, decodes the bytes, converts the image to RGB, and executes inference with the YOLO11 model stored as `est.pt`. Each prediction in the response includes a class label, a confidence score, and the bounding box pixel coordinates of the top-left and bottom-right corners. A detection response is treated as positive by the frontend when at least one prediction carries the label `bulb` above the model's default confidence threshold.

TABLE I
IMPLEMENTED SOFTWARE COMPONENTS

Component	Role in the prototype
React 17	Browser interface, state management, command dispatch
react-webcam	Webcam capture and base64 snapshot encoding
TensorFlow.js Handpose	21-point 3-D hand landmark estimation
Fingerpose 0.1.0	Gesture classification from landmark descriptors
FastAPI	Local HTTP endpoint for YOLO inference requests
YOLO11 (`est.pt`)	Multi-instance bulb detection with bounding boxes
Blynk HTTPS API	Cloud command routing to ESP32 virtual pins
ESP32 + relay	Physical appliance switching per assigned pin

B. Multi-Device Target Selection

The frontend maintains a list of active bounding boxes updated on each 3-second detection cycle. Upon receiving a positive detection response, entries with label `bulb` are retained and their bounding box centers computed as:

$$c_x = \frac{x_1 + x_2}{2}, \qquad c_y = \frac{y_1 + y_2}{2}, \tag{1}$$

where (x_1, y_1) and (x_2, y_2) are the top-left and bottom-right corners, respectively. The list is sorted in ascending order of c_x, and virtual pins are assigned by index position.

When a valid gesture is detected, the wrist landmark coordinates (w_x, w_y) from the most recent Handpose estimate are compared against each stored bounding box center. The target index k is selected as:

$$k = \arg\min_i \sqrt{(w_x - c_{x,i})^2 + (w_y - c_{y,i})^2}. \tag{2}$$

A Blynk HTTPS request is issued to virtual pin Vk with value 1 for `thumbs_up` or value 0 for `thumbs_down`. All other virtual pins receive no update, ensuring unselected appliances are not affected.

C. Gesture Recognition

The gesture recognition component is instantiated only while at least one valid bulb bounding box is stored, enforcing the detection gate. Handpose is loaded once per session and performs landmark estimation on each rendered video frame. Fingerpose evaluates the 21-point array against descriptors for thumbs-up, thumbs-down, raised hand, raised fist, and pointing gestures. A score threshold of 8 is applied; the highest-scoring gesture above this threshold is forwarded to the parent component. Gestures falling below the threshold are silently discarded, providing robustness against ambiguous or transitional hand configurations.

D. IoT Command Dispatch

Fig. 4 illustrates the gesture-to-command mappings. For a recognized `thumbs_up` targeting index k, the frontend issues:

```
GET /external/api/update
    ?token=<DEVICE_TOKEN>&V<k>=1
```

For `thumbs_down`, the same endpoint is called with `V`$\langle k \rangle$`=0`. The device token is read from an environment variable at build time and is not embedded in the compiled JavaScript bundle. The ESP32 firmware monitors the assigned virtual pin via the Blynk library and drives the relay output on each pin value change event.

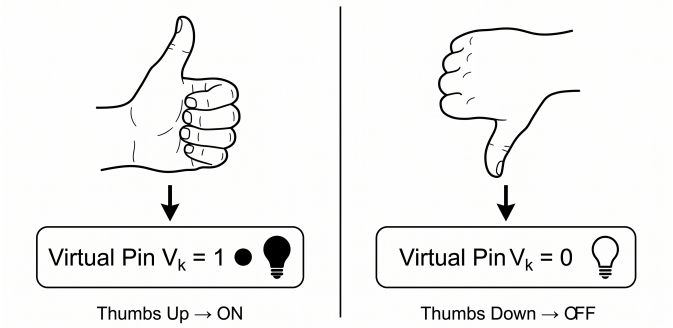

Fig. 4. Gesture-to-command mapping: thumbs-up writes 1 to the target virtual pin; thumbs-down writes 0. Commands are suppressed when no `bulb` detection is active.

V. EVALUATION

A. Test Configurations

The prototype was evaluated across two hardware configurations. In the single-device configuration, one ESP32 module and one relay were connected to a single LED bulb assigned to virtual pin V0. In the two-device configuration, a second independently powered ESP32–relay–bulb assembly was introduced and assigned to virtual pin V1. Both configurations were tested under indoor ambient lighting with the webcam positioned approximately 1 m from the appliances.

B. Single-Device Validation

Three pipeline behaviors were verified systematically. First, the FastAPI endpoint was confirmed to return a `bulb`-class detection when a bulb was present in the camera frame and an empty detection list when the bulb was removed or obscured. Second, the gesture recognition component was observed to remain inactive during periods with no positive detection, confirming correct gate behavior. Third, thumbs-up and thumbs-down gestures were issued across multiple sessions; in each trial the relay transitioned to the corresponding state within the expected round-trip latency window, which includes the Blynk HTTPS request and ESP32 pin polling interval. All three behaviors were confirmed across repeated testing sessions with no observed false switching events during detection-negative intervals.

C. Two-Device Spatial Discrimination

Two concurrently powered bulbs were placed 35 cm apart within the camera field of view. Spatial selection was evaluated by positioning the user's hand near each bulb alternately and issuing thumbs-up and thumbs-down commands. In each trial the relay connected to the targeted bulb changed state in response to the gesture, while the relay of the non-targeted bulb remained unchanged. Selection ambiguity was observed when the horizontal separation between bounding box centers fell below approximately 30 pixels in the camera image, a condition arising when the camera was angled shallowly relative to the plane of the two bulbs. At the 35 cm physical separation and a camera distance of 1 m, this condition was not encountered during testing.

D. Evaluation Protocol for Future Work

A controlled measurement study is required to produce statistically reportable accuracy and latency figures. Table II defines a structured protocol for that study. Detection precision and recall can be measured from annotated video clips spanning varied lighting and partial-occlusion conditions. Gesture command accuracy can be derived from a fixed command sequence with known ground truth. End-to-end latency is measured from gesture onset to confirmed relay state change. Selection accuracy and false command rate complete the multi-device evaluation.

TABLE II
RECOMMENDED EVALUATION PROTOCOL

Metric	Measurement method
Detection precision	Correct detections / all returned detections
Detection recall	Correct detections / all frames with bulb present
Gesture accuracy	Correct ON/OFF commands / all attempted gestures
End-to-end latency	Gesture onset to confirmed relay state change
Selection accuracy	Correct target routed / all multi-device trials
False command rate	Commands issued during detection-negative intervals

VI. DISCUSSION

A. Design Strengths

The principal advantage of the proposed architecture is the directness of the interaction path. The user neither opens an application menu nor searches for a device by name or identifier. Positioning the hand near a visible appliance and forming a gesture is the complete interaction sequence, which closely mirrors the physical act of reaching toward a device. This directness is potentially valuable in accessibility contexts, particularly for users who find touch-based or voice-based interfaces difficult to use reliably.

The spatial selection mechanism addresses a fundamental limitation of camera-based control in multi-device environments. By using the wrist landmark position as an implicit spatial pointer, the system enables target selection without requiring device-specific gestures, voice identifiers, or a calibration phase. The approach requires only that each controlled appliance has a unique Blynk virtual pin, which constitutes a one-time configuration task independent of the number of devices.

The detection gate provides a practical safety property: gesture recognition cannot trigger an actuation event unless the detector has confirmed a valid appliance target in the current frame. This is distinct from systems that process gestures continuously and rely solely on gesture classification accuracy to prevent false actuation.

B. Limitations and Future Directions

The 3-second detection polling interval introduces a lag between the moment an appliance enters the camera frame and the moment gesture recognition becomes active. This delay is a direct consequence of synchronous HTTP polling; a streaming inference architecture or a lighter detection model evaluated at a shorter interval would reduce it substantially. Streaming WebSocket inference would also allow bounding box updates to keep pace with user movement rather than being refreshed at fixed intervals.

Gesture classification accuracy is sensitive to ambient lighting conditions and hand orientation. Smoothing predictions over a sliding window of consecutive frames would reduce transient misclassifications arising from momentary hand pose ambiguity. The Fingerpose score threshold of 8 was set empirically; a systematic calibration study across multiple users and lighting conditions would establish a principled threshold.

The spatial selection step is susceptible to failure when two bounding box centers are close together in the camera coordinate space, as discussed in Section V-B. Incorporating depth information from a stereo or structured-light sensor, or adding a temporal consistency check on the wrist trajectory, would improve robustness in this edge case.

From a deployment perspective, storing Blynk tokens in environment variables rather than source code is a necessary step before any wider use. A production system would additionally require command confirmation feedback to the user, authenticated access to the Blynk project, and relay hardware rated for the mains voltage of the controlled appliances. Extension to additional appliance classes—fans, air conditioners, window actuators—would require either a broader YOLO training set or a separate per-class detection module integrated into the same frontend pipeline.

VII. CONCLUSION

This paper presented a browser-based IoT system for gesture-controlled smart home appliances. The system integrates React, TensorFlow.js Handpose, Fingerpose, a YOLOv11 detector served through FastAPI, and Blynk cloud actuation through ESP32 relay modules into a single, installation-free interaction pipeline. The core contribution is a spatial selection mechanism that assigns each detected appliance bounding box a unique Blynk virtual pin sorted by

horizontal camera position, and routes each gesture command exclusively to the appliance whose bounding box center is nearest the user's wrist landmark, leaving all other connected appliances unaffected. A detection gate suppresses gesture processing when no valid appliance target is present in the frame, reducing the rate of accidental actuation.

Functional testing in single-device and two-device configurations confirmed correct relay switching and successful spatial discrimination at a 35 cm inter-appliance separation. Future work includes a controlled accuracy and latency study with human participants, streaming inference to reduce the detection polling delay, gesture vocabulary expansion for analog appliance control, and in-browser edge deployment of the YOLO model to eliminate the FastAPI dependency. The prototype demonstrates that browser-based computer vision and cloud IoT actuation can be unified into a touch-free, multi-device control pipeline that requires no native application, no wearable sensor, and no dedicated gesture-recognition hardware.

ACKNOWLEDGMENT

This work was carried out at the Military College of Signals, National University of Sciences and Technology, Rawalpindi, Pakistan.

REFERENCES

[1] L. Atzori, A. Iera, and G. Morabito, "The Internet of Things: A survey," *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010.

[2] D. Marikyan, S. Papagiannidis, and E. Alamanos, "A systematic review of the smart home literature: A user perspective," *Technological Forecasting and Social Change*, vol. 138, pp. 139–154, 2019.

[3] R. T. Azuma, "A survey of augmented reality," *Presence: Teleoperators and Virtual Environments*, vol. 6, no. 4, pp. 355–385, 1997.

[4] C.-Y. Yang, Y.-N. Lin, S.-K. Wang, V. R. L. Shen, Y.-C. Tung, F. H. C. Shen, and C.-H. Huang, "Smart control of home appliances using hand gesture recognition in an IoT-enabled system," *Applied Artificial Intelligence*, vol. 37, no. 1, p. 2176607, 2023.

[5] B. I. Alabdullah, H. Ansar, N. A. Mudawi, A. Alazeb, A. Alshahrani, S. S. Alotaibi, and A. Jalal, "Smart home automation-based hand gesture recognition using feature fusion and recurrent neural network," *Sensors*, vol. 23, no. 17, p. 7523, 2023.

[6] TensorFlow, "Face and hand tracking in the browser with MediaPipe and TensorFlow.js," https://blog.tensorflow.org/2020/03/face-and-hand-tracking-in-browser-with-mediapipe-and-tensorflowjs.html, 2020, accessed: 2026-04-28.

[7] P. Milgram and F. Kishino, "A taxonomy of mixed reality visual displays," *IEICE Transactions on Information and Systems*, vol. E77-D, no. 12, pp. 1321–1329, 1994.

[8] M. C. Lopes, P. A. Silva, L. Marenco, E. C. Vilas Boas, J. G. A. de Carvalho, C. A. Ferreira, A. L. O. Carvalho, C. V. R. Guimarães, G. P. Aquino, and F. A. P. de Figueiredo, "GECO: A real-time computer vision-assisted gesture controller for advanced IoT home system," *Sensors*, vol. 26, no. 1, p. 61, 2025.

[9] V. Sideridis, A. Zacharakis, G. Tzagkarakis, and M. Papadopouli, "GestureKeeper: Gesture recognition for controlling devices in IoT environments," in *2019 27th European Signal Processing Conference (EUSIPCO)*, 2019, pp. 1–5.

[10] Fingerpose Contributors, "fingerpose: Finger gesture classifier for hand landmarks," https://www.npmjs.com/package/fingerpose, 2021, accessed: 2026-04-28.

[11] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[12] G. Jocher and J. Qiu, "Ultralytics YOLO11," 2024. [Online]. Available: https://github.com/ultralytics/ultralytics

[13] Blynk, "Update datastream value — Blynk cloud HTTPS API," https://docs.blynk.io/en/blynk.cloud/device-https-api/update-datastream-value, 2026, accessed: 2026-04-28.

[14] Espressif Systems, *ESP32-WROOM-32 Datasheet*, https://documentation.espressif.com/esp32-wroom-32_datasheet_en.html, Espressif Systems, 2024, accessed: 2026-04-28.

[15] Umer481, "Smart-home-IoT-system," https://github.com/Umer481/Smart-Home-IOT-System, 2025, accessed: 2026-04-28.